

Классические задачи синхронизации

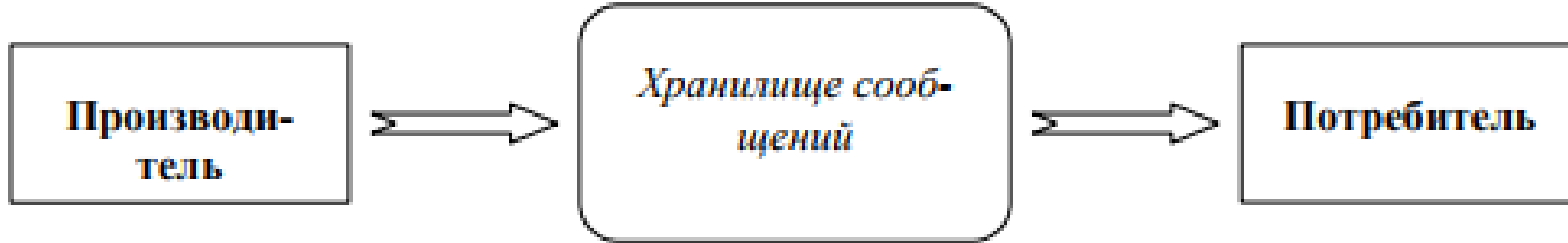
В ходе изучения проблем параллельного программирования и разработки методов их решения сформировался набор некоторых «классических» задач, на которых принято демонстрировать результаты применения новых разрабатываемых подходов.

- Задача «Производители–Потребители» (Producer–Consumer problem);
- Задача «Читатели–Писатели» (Readers-Writers problem);
- Задача «Обедающие философы» (Dining Philosopher problem);
- Задача «Спящий парикмахер» (Sleeping Barber problem)

Задача «Производители–Потребители»

- В наиболее простом случае предполагается, что существует два потока, один из которых (производитель) генерирует сообщения (изделия), а второй поток (потребитель) их принимает для последующей обработки.
- Поток взаимодействует через некоторую область памяти (хранилище), в которой производитель размещает свои генерируемые сообщения и из которой эти сообщения извлекаются потребителем

«Производители–Потребители»



- Хранилище сообщений представляет собой общий разделяемый ресурс, и использование этого ресурса должно быть построено по правилам взаимного исключения.
- Кроме того, следует учитывать, что потребление ресурса иногда может оказаться невозможным (отсутствие сообщений в хранилище), а при добавлении сообщений в хранилище могут происходить задержки (в случае полного заполнения хранилища).

Решение (один из вариантов)

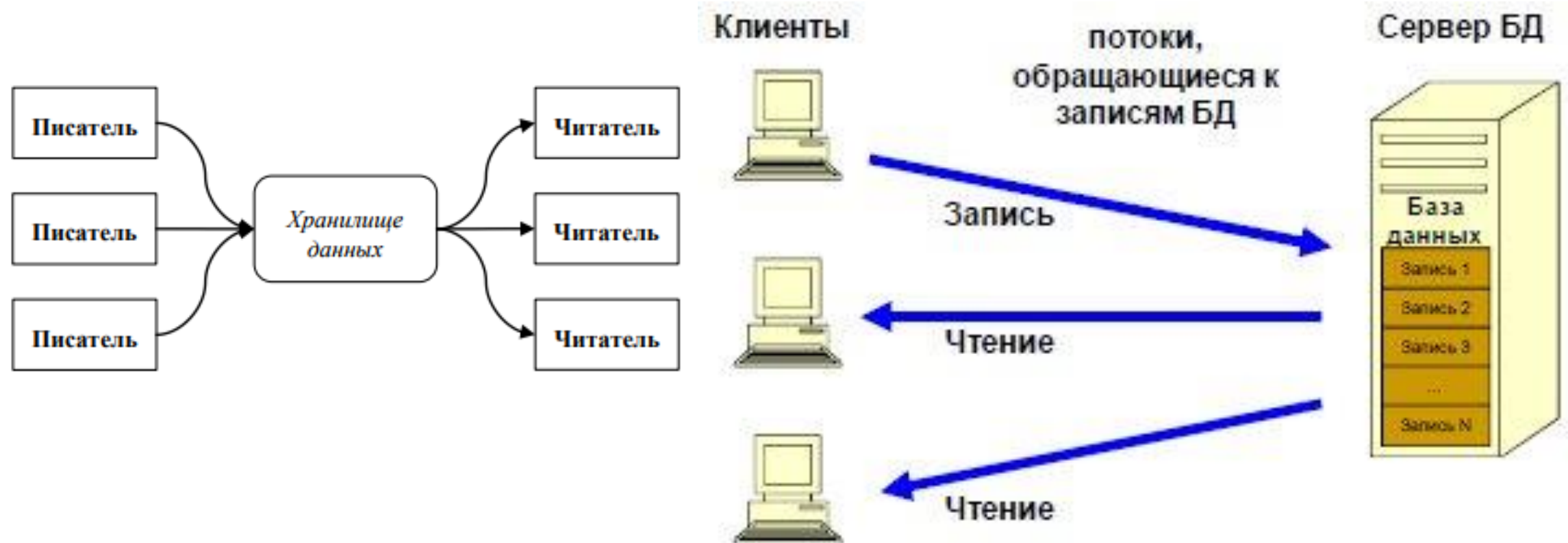
- Для организации работы используем три семафора:
- **Access** – двоичный семафор для организации взаимного исключения при доступе к хранилищу;
- **Full** – общий семафор, блокирующий поток-производитель при попытке записи сообщения в полностью заполненное хранилище (в переменной семафора будет храниться количество имеющихся свободных мест для сообщений в хранилище);
- **Empty** – общий семафор, блокирующий поток-потребитель при попытке чтения сообщения из пустого хранилища (в переменной семафора будет храниться количество имеющихся сообщений в хранилище).

```
// Семафор взаимногоисключения доступа
Semaphore Access = 1;
// Семафор блокировки записи в полное хранилище
Semaphore Full   = n;
// Семафор блокировки чтения из пустого хранилища
Semaphore Empty  = 0;
Producer() {
    <Генерация нового сообщения>
    // Доступ только при наличии пустых мест
    P(Full);
    P(Access); // Блокировка доступа к хранилищу
    <Запись сообщения в хранилище>
    // Снятие блокировки доступа к хранилищу
    V(Access);
    // Отметка наличия сообщений в хранилище
    V(Empty);
}
Consumer() {
    // Доступ только при наличии сообщений
    P(Empty);
    P(Access); // Блокировка доступа к хранилищу
    <Чтение сообщения из хранилища>
    // Снятие блокировки доступа к хранилищу
    V(Access);
    // Отметка наличия пустых мест в хранилище
    V(Full);
    <Обработка полученного сообщения>
}
```

Задача «Читатели–Писатели»

- Пусть имеется некоторая область памяти – хранилище данных, с которым одновременно работают несколько потоков.
- По характеру использования хранилища потоки могут быть разделены на два типа.
- Одна группа – это **потоки-читатели**, которые осуществляют только чтение (без удаления) хранилища.
- Другая – оставшаяся часть потоков – это **потоки-писатели**, которые выполняют запись новых значений имеющихся в хранилище данных

«Читатели–Писатели»



- При рассмотрении этой задачи предполагается, что потоки-читатели не изменяют каких-либо параметров хранилища и могут, тем самым, работать одновременно, не мешая друг другу.
- Для потоков-писателей ситуация обратная – предполагается, что запись не может выполняться несколькими потоками одновременно и, понятно, во время записи не допускаются какие-либо операции чтения.

- Постановка задачи может различаться в правилах разрешения ситуации обращения потока-писателя к хранилищу.
- Если при попытке записи имеются активные потоки-читатели, то поток-писатель должен быть заблокирован до завершения работы потоков-читателей. Вопрос состоит в том, что делать с новыми поступающими запросами на чтение.
- Возможный вариант – отдать предпочтение потокам-читателям (т. е. новые потоки-читатели могут начинать свою работу, не обращая внимания на заблокированный процесс-писатель), однако в этом случае блокировка потока-писателя может продолжаться бесконечно долго.
- Альтернативный подход – блокировка новых потоков-читателей, появившихся после блокировки потока-писателя.

Принципиальная реализация задачи «Читатели – писатели»



Писатель

- Дождаться освобождения хотя бы одной ячейки буфера
- Захватить буфер
- Записать данные в первую свободную ячейку
- Освободить буфер
- Сообщить о том, что в буфере появилась новая информация

Читатель

- Дождаться появления в буфере хотя бы одной записанной ячейки
- Захватить буфер
- Прочитать данные из буфера и освободить ячейку
- Освободить буфер
- Сообщить о появлении в буфере свободной ячейки

Решение

- **ReadCount** – переменная-счетчик количества активных потоков-читателей;
- **ReadSem** – двоичный семафор для взаимного исключения доступа к переменной ReadCount;
- **Access** – двоичный семафор для организации взаимного исключения при доступе к хранилищу.

```

// Счетчик количества активных потоков-читателей
int ReadCount = 0;
// Семафор доступа к переменной ReadCount
Semaphore ReadSem = 1;
// Семафор доступа к хранилищу
Semaphore Access = 1;
Writer() { // Поток-писатель
    // Блокировка доступа к хранилищу
    P(Access);
    <Выполнение операции записи>
    // Снятие блокировки доступа к хранилищу
    V(Access);
}
Reader() { // Поток-читатель
    // Блокировка доступа к переменной ReadCount
    P(ReadSem);
    // Изменение счетчика активных читателей
    ReadCount++;
    if( ReadCount == 1 )
        // Блокировка доступа к хранилищу
        // (если поток-читатель первый)
        P(Access);
    // Снятие блокировки доступа к ReadCount
    V(ReadSem);
    <Выполнение операции чтения>
    // Блокировка доступа к переменной ReadCount
    P(ReadSem);
    // Изменение счетчика активных читателей
    ReadCount--;
    // Снятие блокировка доступа к хранилищу
    // (если завершается последний поток-читатель)
    if( ReadCount == 0 )
        V(Access);
    // Снятие блокировки доступа к ReadCount
    V(ReadSem);
}

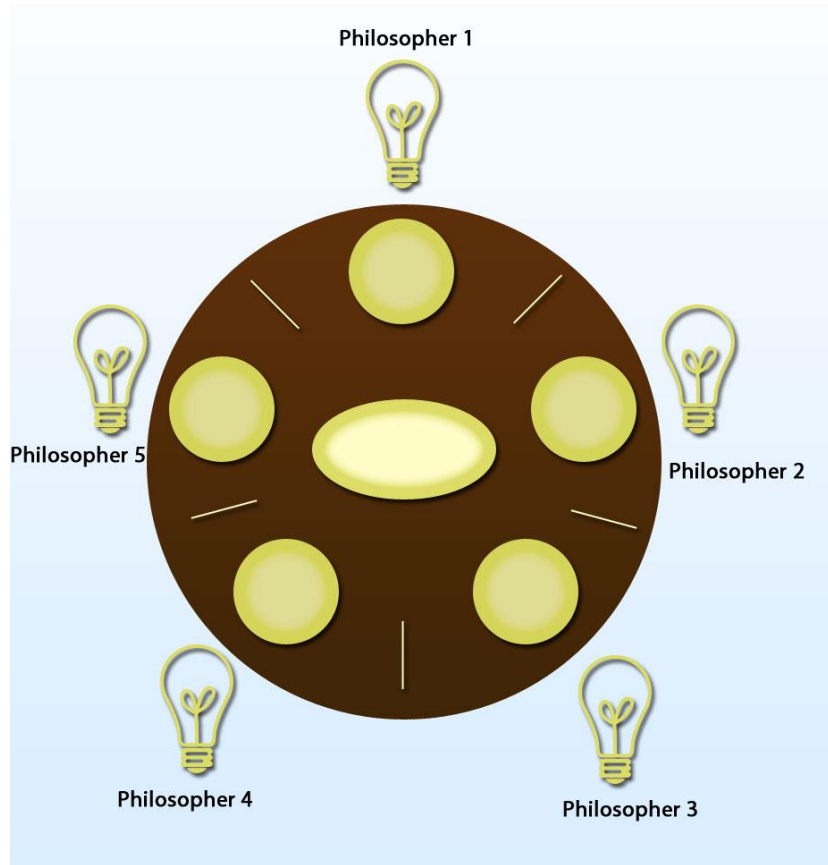
```

Задача «Обедающие философы»

- Если задача «Читатели–Писатели» помогает демонстрировать методы параллельного и исключительного доступа к одному общему ресурсу, то задача «Обедающие философы» позволяет рассмотреть способы доступа нескольких потоков к нескольким разделяемым ресурсам.
- Исходная формулировка задачи впервые предложена Э. Дейкстрой.

Формулировка задачи

- Представляется ситуация, в которой пять философов располагаются за круглым столом.
- При этом философы либо размышляют, либо кушают.
- Для приема пищи в центре стола большое блюдо с неограниченным количеством спагетти, и тарелки, по одной перед каждым философом.
- Предполагается, что поесть спагетти можно только с использованием двух вилок. Для этого на столе располагается ровно пять вилок – по одной между тарелками философов.



- Для того, чтобы приступить к еде, философ должен взять вилки слева и справа (если они не заняты), наложить спагетти из большого блюда в свою тарелку, поесть, а затем обязательно положить вилки на свои места для их повторного использования (проблема чистоты вилок в задаче не рассматривается).
- Нетрудно заметить, что в данной задаче философы представляют собой потоки, а вилки – общие разделяемые ресурсы.


```
// Семафоры доступа к вилкам
Semaphore fork[5] = { 1, 1, 1, 1, 1 };
// Поток -философ (для всех философов одинаковый)
Prilosopher() {
    // i - номер философа
    while (1) {
        P(fork[i]);           // Доступ к левой вилке
        P(fork[(i+1)%5]);     // Доступ к правой вилке
        <Питание>
        // Освобождение вилок
        V(fork[i]); V(fork[(i+1)%5])
        <Размышление>
    }
}
```

- Можно увидеть, что данное решение может приводить к тупиковым ситуациям – например, когда все философы одновременно проголодаются и каждый из них возьмет свои левые вилки.
- В результате правые вилки для всех философов окажутся занятыми и философы перейдут к бесконечному ожиданию (сложно выявить подобную ситуацию при помощи тестов; кроме того, подобную ошибочную ситуацию сложно повторить при повторных запусках программы).

```
// Семафоры доступа к вилкам
Semaphore fork[5] = { 1, 1, 1, 1, 1 };
// Поток-философ (для всех, кроме четвертого)
Prilosopher() {
    // i - номер философа
    while (1) {
        P(fork[i]);          // Доступ к левой вилке
        P(fork[(i+1)%5]);    // Доступ к правой вилке
        <Питание>
        // Освобождение вилок
        V(fork[i]); V(fork[(i+1)%5])
        <Размышление>
    }
Prilosopher4() { // Поток для четвертого философа)
    while (1) {
        P(fork[0]); // Доступ к правой вилке
        P(fork[4]); // Доступ к левой вилке
        <Питание>
        V(fork[0]); V(fork[4]) // Освобождение вилок
        <Размышление>
    }
}
```

Задача «Спящий парикмахер»

- Данная задача также в числе широко используемых примеров для демонстрации проблем синхронизации.
- На примере этой задачи можно показать методы последовательного доступа к набору разделяемых ресурсов и рассмотреть организацию вычислений в соответствии со схемой «клиент–сервер».

Общая схема задачи «Спящий парикмахер»



Постановка задачи

- В парикмахерской имеется два помещения: комната ожидания, в которой ограниченное количество мест, и рабочая комната с единственным креслом, в котором располагается обслуживаемый клиент.
- Посетители заходят в парикмахерскую – если комната ожидания заполнена, то поворачиваются и уходят; иначе занимают свободные места и засыпают, ожидая своей очереди к парикмахеру.
- Парикмахер, если есть клиенты, приглашает одного из них в рабочую комнату и подстригает его.
- После стрижки клиент покидает парикмахерскую, а парикмахер приглашает следующего посетителя и т. д.
- Если клиентов нет (комната ожидания пуста), парикмахер садится в свое рабочее кресло и засыпает. Будит его очередной появляющийся посетитель парикмахерской.

- В данной задаче ресурсами являются места ожидания и рабочее кресло.
- Потоки-клиенты должны получать эти ресурсы строго последовательно: сначала посетитель должен найти место в комнате ожидания и только затем занять очередь к парикмахеру.
- При этом предоставление рабочего кресла для обслуживания производит специальный процесс-парикмахер.
- В этом плане, парикмахера можно интерпретировать как сервер, предоставляющий требуемый сервис.

События

- **Client** – событие, означающее, что есть ожидающие посетители; событие объявляется каждый раз при появлении нового клиента; данное событие пробуждает спящего парикмахера, заснувшего при отсутствии клиентов;
- **Barber** – событие при освобождении парикмахера; по данному событию пробуждается один из ожидающих клиентов, который и переходит в рабочую комнату для обслуживания;
- **Service** – событие при завершении обслуживания очередного клиента; клиент может покинуть парикмахерскую.


```
Monitor BarberShop {  
    // Максимальное количество посетителей  
    const int ClientMax = 10;  
    // Текущее количество клиентов  
    int ClientNum = 0;  
    // Событие: Имеются ожидающие посетители  
    cond Client;  
    // Событие: Парикмахер свободен  
    cond Barber;  
    // Событие: Обслуживание завершено  
    cond Service;
```

```
Client() { // Поток клиента
    // Ждать только если есть места
    if ( ClientNum < ClientMax ) {
        ClientNum++;
        // Есть посетители -разбудить парикмахера
        signal(Client);
        wait(Barber);    // Ждать парикмахера
        wait(Service);   // Ждать окончания стрижки
    }
}
```

```
Barber() { // Поток парикмахера
    while (1) {
        // Ждать посетителей (спать)
        if ( ClientNum == 0 ) wait(Client);
        // Парикмахер свободный - пригласить клиента
        signal(Barber);
        ClientNum--;
        <Обслуживание>
        // Обслуживание завершено - клиент может уйти
        signal(Service);
    }
}
}
```